

Playwright Agentic AI Framework Guide

Who This Guide Is For

This guide explains the Playwright Agentic AI Framework in a simple, practical way. It is written for:

- new automation engineers joining the project
- QA leads reviewing the architecture
- developers who want to understand UI/API test flow
- AI-agent users who need to understand spec-driven generation
- teams who want to use this framework as a reference architecture

The goal is simple:

Understand where input starts, how AI converts it into tests, how Playwright runs it, and how the framework remembers, validates, reports, and improves over time.

1. Project Architecture

The project follows a framework-first, manifest-driven, validator-governed architecture.

High-level layers:

```
Input Layer
  specs/

AI Governance Layer
  src/ai/

Orchestration Layer
  scripts/

Execution Framework Layer
  src/fixtures
  src/pages
  src/components
  src/api
  src/config
  src/utils

Executable Test Layer
  tests/

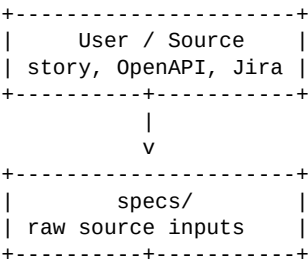
Evidence And Feedback Layer
  artifacts/
```

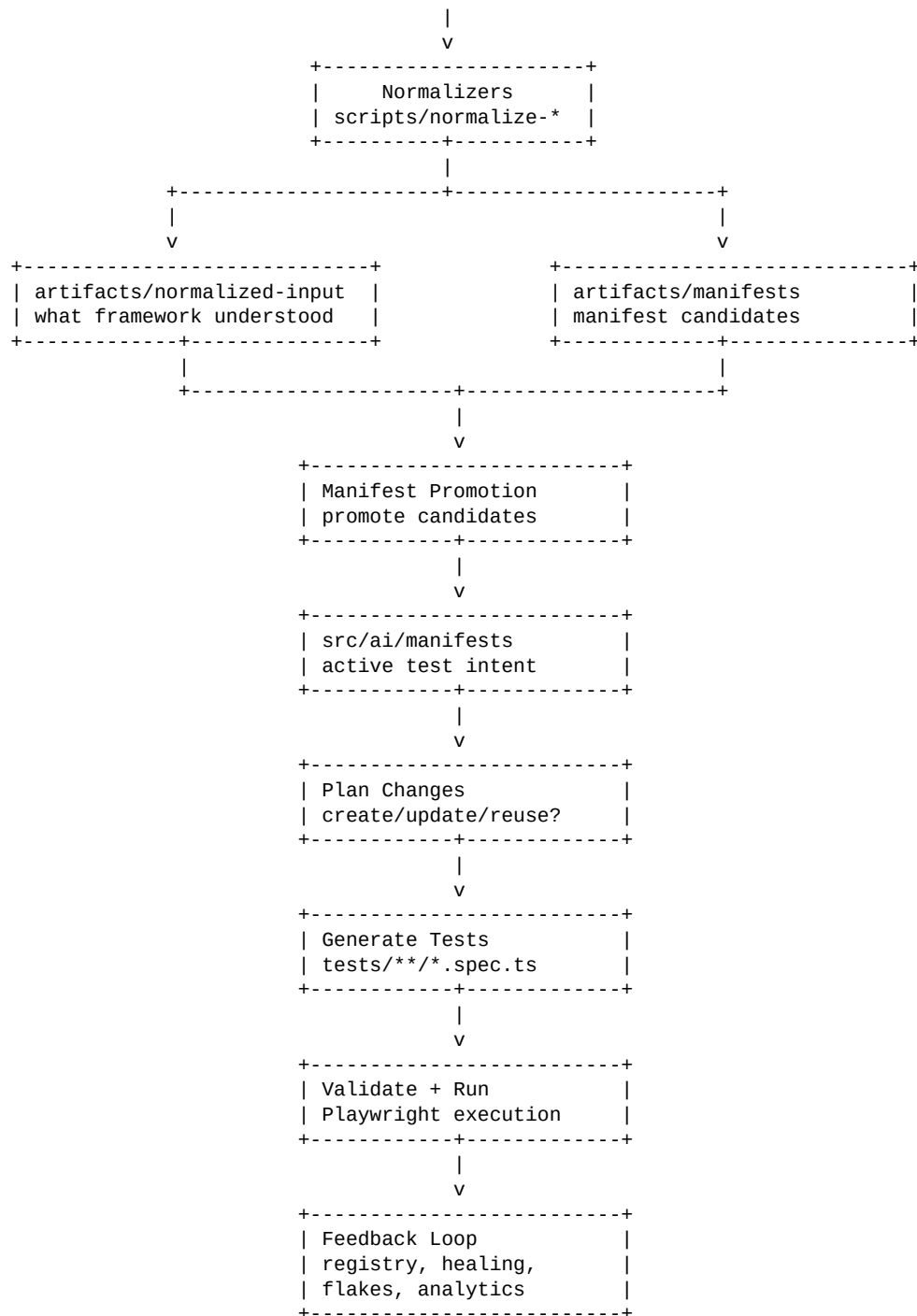
The most important architecture idea:

AI does not directly write random Playwright tests.
AI must go through normalized input, manifests, planning, generation, validation, and registry traceability.

This makes the framework safer, repeatable, and easier to maintain in a product-company setup.

2. Architecture Diagram





3. End To End Framework Flow

The complete framework flow is:

1. Put source input under specs/
2. Normalize source input into structured JSON
3. Create manifest candidates
4. Promote approved candidates into active manifests
5. Plan changes by comparing manifests, registry, and test files
6. Generate or reuse Playwright tests
7. Sync registry
8. Validate manifests, registry, code, framework rules, and secrets
9. Run Playwright tests
10. Update registry execution status
11. Classify failures
12. Create healing plan
13. Analyze flakes
14. Generate execution analytics and reports

Short memory version:

specs -> normalized input -> candidates -> active manifests -> plan -> tests -> registry -> run
-> feedback

Daily command example:

```
npm run agentic -- --action=full --input-type=openapi --mode=api --strategy=auto --env=qa
```

This means:

Use OpenAPI input.
Promote manifest candidates.
Plan and generate API tests.
Validate the framework.
Run API tests.
Generate reports and feedback artifacts.

4. Framework Components

The framework has these major components.

Input Components

specs/openapi
specs/stories
specs/jira
specs/user-input
specs/screenshots

These folders contain raw inputs. They are not executable tests.

AI Governance Components

src/ai/agent.md
src/ai/governance
src/ai/manifests
src/ai/registry
src/ai/schemas
src/ai/skills
src/ai/templates

These files define how AI is allowed to work.

Orchestration Components

scripts/agentic-runner.mjs
scripts/normalize-openapi.mjs
scripts/normalize-story.mjs
scripts/normalize-jira.mjs
scripts/normalize-user-input.mjs
scripts/normalize-screenshot.mjs
scripts/promote-manifest-candidates.mjs
scripts/plan-changes.mjs
scripts/generate-from-manifests.mjs
scripts/sync-registry.mjs
scripts/validate-manifests.mjs
scripts/check-registry.mjs
scripts/validate-framework.mjs
scripts/validate-secrets.mjs
scripts/classify-failures.mjs
scripts/heal-failures.mjs
scripts/analyze-flakes.mjs
scripts/generate-execution-analytics.mjs

These scripts are the automation pipeline.

Runtime Framework Components

src/config/env.ts
src/fixtures/base.fixture.ts
src/pages
src/components
src/api
src/utils
playwright.config.ts

These files make the tests run.

Test Components

```
tests/ui
tests/api
tests/hybrid
```

These are executable Playwright tests.

Evidence Components

```
artifacts/normalized-input
artifacts/manifests
artifacts/plans
artifacts/generated
artifacts/reports
artifacts/execution
artifacts/healing
artifacts/flakes
artifacts/analytics
artifacts/security
artifacts/validator
artifacts/registry-snapshots
```

Artifacts are evidence. They show what the framework understood, planned, generated, executed, and learned.

5. Spec Driven AI Workflow

Spec-driven means tests start from a source specification, not from random code.

Supported inputs:

- OpenAPI YAML/JSON
- Story markdown
- Jira markdown/JSON
- User instruction snapshot
- Screenshot image or metadata

The AI/spec workflow is:

```
Raw input
-> normalized JSON
-> manifest candidate
-> active manifest
-> change plan
-> generated/reused test
-> registry sync
-> validation
-> execution
-> feedback loop
```

Example Story Input

```
specs/stories/story-test-automation-playground.md
```

Example content:

As a test automation engineer, I want the Test Automation Playground home page to load so that UI scenarios can be generated from stable practice flows.

Step 1: Normalize

Script:

```
scripts/normalize-story.mjs
```

Outputs:

```
artifacts/normalized-input/story-normalized.json
artifacts/manifests/story-manifest-candidates.json
```

Normalized JSON means:

What did the framework understand from the story?

Manifest candidate means:

What test scenario could be created from that understanding?

Step 2: Promote

Script:

```
scripts/promote-manifest-candidates.mjs
```

Output:

```
src/ai/manifests/*.json
artifacts/plans/manifest-promotion-summary.json
```

Important difference:

```
artifacts/manifests = draft candidates
src/ai/manifests = active manifests used for generation
```

Step 3: Plan

Script:

```
scripts/plan-changes.mjs
```

Output:

```
artifacts/plans/change-plan.json
```

The plan decides:

```
create - new generated test needed
update - generated test can be regenerated
reuse  - manual/framework-owned test exists, do not overwrite
stale   - registry has a scenario but no matching manifest
broken  - manifest, registry, and files disagree
```

Manifest says what should exist. Plan decides what is safe to change.

Step 4: Generate

Script:

```
scripts/generate-from-manifests.mjs
```

Outputs:

```
tests/ui/**/*.spec.ts
tests/api/**/*.spec.ts
tests/hybrid/**/*.spec.ts
artifacts/generated/generation-summary.json
```

Generated tests are marked:

```
// AUTO-GENERATED by manifest generator. Do not edit directly.
// scenarioId: API_USERS_GET_001
// sourceRef: dummyjson-users.yaml#getUsers
```

Manual tests without this header are protected.

Step 5: Registry

Script:

```
scripts/sync-registry.mjs
```

Updates:

```
src/ai/registry/test-registry.json
```

Registry remembers:

```
scenarioId
sourceRef
filePath
layer
owner
generatedBy
```

```
lastValidatedAt
lastExecutedAt
lastStatus
healingHistory
```

Step 6: Validation

Command:

```
npm run validate
```

It runs:

```
typecheck
validate manifests
check registry
validate framework governance
validate secrets
```

Step 7: Execution And Feedback

After Playwright runs, the framework produces:

```
artifacts/reports/results.json
artifacts/reports/results.md
artifacts/healing/failure-classification.json
artifacts/healing/healing-plan.json
artifacts/flakes/flake-analysis.json
artifacts/analytics/execution-analytics.json
```

This is the feedback loop.

6. Flow Diagram

```
specs/
|
| raw source input
v
scripts/normalize-*.mjs
|
| creates normalized JSON and candidates
v
artifacts/normalized-input/*.json
artifacts/manifests/*-manifest-candidates.json
|
| promote candidates
v
src/ai/manifests/*.json
|
| plan safe file changes
v
artifacts/plans/change-plan.json
|
| generate tests
v
tests/**/*spec.ts
|
| sync traceability
v
src/ai/registry/test-registry.json
|
| validate and execute
v
artifacts/reports/results.json
|
| learn from execution
v
artifacts/healing
artifacts/flakes
artifacts/analytics
```

7. Complete UI Flow

UI flow in this framework:

```
UI test file
-> base fixture
-> page object
-> component object
-> Playwright browser action
-> assertion
```

UI Flow Diagram

```
tests/ui/smoke/playground-home.smoke.spec.ts
|
v
src/fixtures/base.fixture.ts
|
| provides playgroundPage
v
src/pages/app/test-automation-playground.page.ts
|
| calls reusable component classes
v
src/components/*.component.ts
|
v
Playwright Page / Locator APIs
```

UI Test Example

```
import { test, expect } from '@fixtures/base.fixture';

test.describe('@ui @smoke test automation playground', () => {
  test('[ui] UI_PLAYGROUND_HOME_001 - playground home loads', async ({ playgroundPage, page }) => {
    {
      await playgroundPage.open();
      await playgroundPage.expectLoaded();

      await expect(page.locator('body')).toContainText(/test|automation|playground/i);
    }
  });
});
```

Fixture Example

```
playgroundPage: async ({ page }, use) => {
  await use(new TestAutomationPlaygroundPage(page));
}
```

The test does not create the page object manually. The fixture gives it.

Page Object Example

```
export class TestAutomationPlaygroundPage extends BasePage {
  readonly dialogs: BrowserDialogsComponent;
  readonly files: FileOperationsComponent;

  constructor(page: Page) {
    super(page);
    this.dialogs = new BrowserDialogsComponent(page);
    this.files = new FileOperationsComponent(page);
  }

  async open(): Promise<void> {
    await this.page.goto('/', { waitUntil: 'commit' });
  }

  async verifyAlerts(): Promise<void> {
    await this.openArea('alerts');
    await this.expectAreaTitle('Alerts Testing');

    const message = await this.dialogs.acceptAlert(async () => {
      await this.page.locator('#alert-btn').click({ noWaitAfter: true });
    });

    expect(message.length).toBeGreaterThan(0);
  }
}
```

Component Object Example

```
export class BrowserDialogsComponent {
  constructor(private readonly page: Page) {}

  async acceptAlert(trigger: () => Promise<void>): Promise<string> {
    const dialogPromise = this.page.waitForEvent('dialog');
    const triggerPromise = trigger();
    const dialog = await dialogPromise;
    const message = dialog.message();
    await dialog.accept();
    await triggerPromise;
    return message;
  }
}
```

UI Responsibility Split

Test file:

says what scenario is being tested

Fixture:

creates page object

Page object:

owns page-level business flow

Component object:

owns reusable UI operation like dialog, table, upload, widgets

Playwright:

executes browser actions

If the application has multiple pages, create multiple page objects:

```
src/pages/app/login.page.ts
src/pages/app/dashboard.page.ts
src/pages/app/orders.page.ts
```

Then add them to:

```
src/fixtures/base.fixture.ts
```

8. Complete API Flow

API flow in this framework:

API test file

```
-> base fixture
-> API client
-> ResourceClient
-> BaseApiClient
-> Playwright APIRequestContext
-> response assertion
```

API Flow Diagram

```
tests/api/smoke/users.smoke.spec.ts
|
v
src/fixtures/base.fixture.ts
|
| provides usersClient
v
src/api/clients/users.client.ts
|
v
src/api/core/resource.client.ts
|
v
src/api/core/base-api.client.ts
|
v
Playwright APIRequestContext
```


API Test Example

```
import { test, expect } from '@fixtures/base.fixture';
import { pagedUsersSchema } from '@api/contracts/dummyjson.contracts';
import { expectJsonResponse } from '@utils/assertions';
import { expectToMatchSchema } from '@utils/api-contract';

test.describe('@api @smoke users', () => {
  test('[api] API_USERS_GET_001 - list /users matches source contract', async ({ usersClient })
  => {
    const response = await usersClient.list({ limit: 10 });
    await expectJsonResponse(response, 200);

    const body = await response.json();
    expectToMatchSchema(body, pagedUsersSchema);
    expect(Array.isArray(body.users)).toBe(true);
    expect(body.users.length).toBeGreaterThan(0);
  });
});
```

API Fixture Example

```
apiContext: async ({ runtime }, use) => {
  const apiContext = await request.newContext({
    baseUrl: runtime.apiBaseUrl,
    timeout: runtime.requestTimeoutMs,
    extraHTTPHeaders: {
      'Content-Type': 'application/json'
    }
  });
},

await use(apiContext);
await apiContext.dispose();
},

usersClient: async ({ apiContext }, use) => {
  await use(new UsersClient(apiContext));
}
```

Users Client Example

```
export class UsersClient extends ResourceClient {
  constructor(request: APIRequestContext) {
    super('/users', request);
  }

  async list(query: PageQuery | number = { limit: 10 }): Promise<APIResponse> {
    return typeof query === 'number' ? super.list({ limit: query }) : super.list(query);
  }
}
```

Resource Client Example

```
export abstract class ResourceClient extends BaseApiClient {
  protected constructor(protected readonly resourcePath: string, requestContext:
  APIRequestContext) {
    super(requestContext);
  }

  list(query: PageQuery = {}): Promise<APIResponse> {
    return this.get(this.resourcePath, { ...query });
  }

  getById(id: number): Promise<APIResponse> {
    return this.get(`${this.resourcePath}/${id}`);
  }
}
```

Base API Client Example

```
export abstract class BaseApiClient {
  protected constructor(protected readonly request: APIRequestContext) {}

  protected get(path: string, params?: QueryParams, headers?: Record<string, string>):
  Promise<APIResponse> {
    return this.request.get(path, { params: this.cleanParams(params), headers });
  }
```

```
}  
}
```

API Responsibility Split

Test:
 validates scenario

Fixture:
 creates API context and clients

Domain client:
 exposes business API methods like usersClient.list()

ResourceClient:
 owns common REST actions like list, getById, create, update, delete

BaseApiClient:
 owns low-level HTTP get, post, put, patch, delete

APIRequestContext:
 executes actual HTTP request

For ecommerce, examples would be:

```
OrdersClient  
PaymentsClient  
CheckoutClient  
ProductsClient  
CartClient
```

Test chooses the client:

```
test('get order by id', async ({ ordersClient }) => {  
  const response = await ordersClient.getByOrderId(12345);  
});
```

9. Environment Config Flow

Environment config is handled by:

src/config/env.ts

Supported real files:

```
.env  
.env.local  
.env.qa  
.env.stage
```

Example files:

```
.env.local.example  
.env.qa.example  
.env.stage.example
```

Runtime loading order:

1. Load .env
2. Resolve TEST_ENV
3. Load .env.<TEST_ENV>
4. Shell or CI variables override file values
5. Return typed RuntimeConfig

Example commands:

```
TEST_ENV=qa npm run test:api  
npm run agentica -- --action=run --mode=ui --env=stage  
npm run agentica -- --action=run --mode=api --test-env=qa
```

UI Config Usage

```
playwright.config.ts  
-> loadRuntimeConfig()  
-> use.baseUrl = runtime.uiBaseUrl
```

So this:

```
await page.goto('');
```

opens:

```
runtime.uiBaseUrl + '/'
```

API Config Usage

```
base.fixture.ts
-> runtime fixture
-> apiContext baseUrl = runtime.apiBaseUrl
-> API clients use relative paths
```

So this:

```
await usersClient.list({ limit: 10 });
```

calls:

```
runtime.apiBaseUrl + '/users?limit=10'
```

10. Feedback Loop

The feedback loop starts after test execution.

```
Playwright results
-> registry update
-> failure classification
-> healing plan
-> flake analysis
-> execution analytics
-> markdown reports
```

Scripts:

```
scripts/update-registry-from-results.mjs
scripts/classify-failures.mjs
scripts/heal-failures.mjs
scripts/analyze-flakes.mjs
scripts/generate-execution-analytics.mjs
scripts/generate-md-report.mjs
```

Outputs:

```
src/ai/registry/test-registry.json
artifacts/execution/execution-registry-update.json
artifacts/healing/failure-classification.json
artifacts/healing/healing-plan.json
artifacts/flakes/flake-analysis.json
artifacts/analytics/execution-analytics.json
artifacts/reports/results.md
```

Why it matters:

```
Normal framework:
  Run tests -> pass/fail
```

```
Agentic framework:
  Run tests -> understand result -> classify failure -> gate healing -> learn from execution
```

Healing is proposal-first. The framework does not automatically hide product bugs.

11. Key Terms

Spec

Raw requirement source. Example: OpenAPI, story, Jira, screenshot.

Normalized Input

Structured JSON that says what the framework understood.

Manifest Candidate

Draft machine-readable test plan.

Active Manifest

Approved manifest under src/ai/manifests. This drives generation.

Change Plan

Safety decision before generation. It decides create, update, reuse, stale, or broken.

Registry

Traceability memory. It maps scenarioId, sourceRef, filePath, layer, and execution status.

Page Object

Class that owns page-level UI behavior.

Component Object

Reusable UI behavior class such as dialogs, files, tables, widgets.

API Client

Domain-specific API wrapper such as UsersClient or ProductsClient.

ResourceClient

Reusable REST resource operations.

BaseApiClient

Low-level HTTP wrapper around Playwright APIRequestContext.

Feedback Loop

Post-run process that updates registry, classifies failures, creates healing plans, detects flakes, and generates analytics.

12. Important Points To Remember

If you remember only five things, remember these:

1. specs/ is input
2. src/ai/manifests is active intent
3. tests/ contains executable Playwright specs
4. src/ai/registry is traceability memory
5. artifacts/ contains evidence, reports, and feedback

Daily memory:

```
specs = input
src/ai = brain
tests = execution
registry = memory
artifacts = evidence
```

Thin test rule:

```
UI tests call page objects.
Page objects call component objects.
API tests call API clients.
API clients call ResourceClient/BaseApiClient.
```

AI governance rule:

```
AI can generate only inside approved contracts.
Manual/framework-owned tests must not be overwritten.
```

13. What This Framework Achieves

This framework solves several real-world automation problems.

Problem: AI generates messy tests

Solution:

Manifest-driven generation
Governance rules
Registry traceability
Validation gates

Problem: Tests become hard to maintain

Solution:

Thin tests
Page Object Model
Component Object Model
API client wrappers
Shared fixtures

Problem: Environment data is inconsistent

Solution:

.env
.env.local
.env.qa
.env.stage
Typed RuntimeConfig

Problem: Failures are not understood

Solution:

Failure classification
Healing gate
Healing plan
Flake analysis
Execution analytics

Problem: Generated tests lose traceability

Solution:

scenarioId
sourceRef
test-registry.json
registry snapshots

Problem: API tests only check status code

Solution:

AJV schema validation
API contracts
Reusable assertion helpers

14. Why This Is A Strong Production-Grade Reference

This project is strong because it combines:

- Playwright execution
- TypeScript strict typing
- Page and component objects
- API client abstraction
- environment-specific runtime config
- spec-driven AI generation
- governance rules
- validation gates
- traceability registry
- feedback loop

- security validation
- flake and analytics reporting

Most Playwright projects have tests and page objects.

This project adds:

AI safety
spec-driven generation
registry memory
post-run intelligence
controlled healing

That is what makes it a strong reference architecture.

Honest maturity view:

Production-grade framework foundation: yes
Strong reference architecture: yes
Enterprise platform with years of execution history: not yet

15. Folder And File Explanation

Root Files

README.md

Project overview, commands, and quick start.

CONTRIBUTING.md

Rules for contributors and safe framework changes.

package.json

NPM commands for tests, validation, agentic workflow, reports, security, analytics.

playwright.config.ts

Playwright runtime configuration, projects, reports, trace, screenshot, video, baseURL.

playwright.auth.config.ts

Separate auth setup config.

tsconfig.json

TypeScript config, strict mode, path aliases.

eslint.config.mjs

Linting rules.

.env.example
.env.local.example
.env.qa.example
.env.stage.example

Environment config templates.

specs/

specs/openapi

OpenAPI specs for API generation.

specs/stories

Story markdown files for UI generation.

specs/jira

Jira markdown or JSON exports.

specs/user-input

Prompt or user instruction snapshots.

specs/screenshots

Screenshot images or metadata for visual/user-flow inputs.

src/ai/

src/ai/agent.md

Main agent behavior contract.

src/ai/governance/generation-rules.md

Rules for file naming, test titles, generation style.

src/ai/governance/framework-ownership.md

Defines files and folders the framework owns.

src/ai/governance/validation-rules.md

Defines what should block bad generated output.

src/ai/governance/healing-rules.md

Defines safe and unsafe healing behavior.

src/ai/governance/runtime-governance.md

Defines runtime artifacts, reporting, failure classification, flake analysis.

src/ai/manifests

Active scenario manifests. These drive test generation.

src/ai/registry/test-registry.json

Traceability and execution memory.

src/ai/schemas

JSON schemas for manifests and registry.

src/ai/templates

Generation templates/guides.

src/ai/skills

Agent skill instructions.

src/fixtures/

src/fixtures/base.fixture.ts

Main test fixture. Provides runtime, apiContext, API clients, page objects, and testData.

src/fixtures/role.fixture.ts

Role/storage-state helper fixture.

src/pages/

src/pages/common/base.page.ts

Base page class.

src/pages/app/test-automation-playground.page.ts

Main UI page object. Owns page-level flows such as verifyForms, verifyAlerts, verifyFiles.

src/components/

Reusable UI operation classes:

browser-dialogs.component.ts

file-operations.component.ts

form-controls.component.ts

frames-windows.component.ts

interactions.component.ts

table.component.ts

widgets.component.ts

dynamic-content.component.ts

shadow-dom.component.ts

These avoid stuffing all UI logic into one page object.

src/api/

`src/api/core/base-api.client.ts`

Low-level HTTP methods.

`src/api/core/resource.client.ts`

Common REST resource operations.

`src/api/clients`

Domain API clients: users, auth, products, carts, posts.

`src/api/contracts`

API response schemas for contract validation.

`src/api/models`

API response TypeScript models.

src/config/

`src/config/env.ts`

Loads `.env` and `.env.<TEST_ENV>`, validates runtime config.

`src/config/constants.ts`

Shared constants such as tags and storage-state paths.

src/utils/

`assertions.ts`

Reusable test assertions.

`api-contract.ts`

AJV schema assertion helper.

`test-data-manager.ts`

Cleanup task manager for test data lifecycle.

`random.ts`

`naming.ts`

`idempotency.ts`

Reusable data and naming helpers.

tests/

`tests/ui`

UI tests.

`tests/api`

API tests.

`tests/hybrid`

Tests that combine API setup and UI verification.

scripts/

The scripts folder is the automation engine.

Important scripts:

`agentic-runner.mjs`

Main orchestrator.

`normalize-*.mjs`

Convert raw input into normalized JSON and manifest candidates.

`promote-manifest-candidates.mjs`

Moves approved candidates into active manifests.

`plan-changes.mjs`

Determines create/update/reuse/stale/broken.

`generate-from-manifests.mjs`

Generates Playwright tests from active manifests.

`sync-registry.mjs`

Updates registry traceability.

`validate-*.mjs`

`check-registry.mjs`

Validation and governance checks.

`classify-failures.mjs`

`heal-failures.mjs`

`analyze-flakes.mjs`

`generate-execution-analytics.mjs`

Feedback loop scripts.

artifacts/

Artifacts are generated evidence.

`artifacts/normalized-input`

What the framework understood from input.

`artifacts/manifests`

Manifest candidates.

`artifacts/plans`

Change plans and promotion summaries.

`artifacts/generated`

Generation summary.

`artifacts/reports`

Playwright JSON and markdown reports.

`artifacts/healing`

Failure classification and healing plan.

`artifacts/flakes`

Flake analysis.

`artifacts/analytics`

Execution analytics.

`artifacts/security`

Secret validation output.

`artifacts/validator`

Framework validation output.

16. New Person Learning Path

If a new person joins the project, teach them in this order.

Step 1: Understand the simple memory model

`specs = input`

`src/ai = brain`

```
tests = execution
registry = memory
artifacts = evidence
```

Step 2: Understand UI execution

```
test -> fixture -> page object -> component object -> browser action
```

Step 3: Understand API execution

```
test -> fixture -> API client -> ResourceClient -> BaseApiClient -> HTTP request
```

Step 4: Understand spec-driven generation

```
input -> normalized JSON -> candidate manifest -> active manifest -> plan -> generated test
```

Step 5: Understand validation

```
npm run validate
```

This protects the framework.

Step 6: Understand feedback

```
results -> registry update -> failure classification -> healing plan -> flakes -> analytics
```

Step 7: Make a safe change

Small practice:

1. Add a story under specs/stories
2. Run normalize
3. Review candidate manifest
4. Promote
5. Plan
6. Generate
7. Validate
8. Run
9. Review reports

17. Final Cheat Sheet

Command:

```
npm run agentic -- --action=full --input-type=openapi --mode=api --strategy=auto --env=qa
```

Input:

```
specs/
```

Normalized:

```
artifacts/normalized-input
```

Candidates:

```
artifacts/manifests
```

Active manifests:

```
src/ai/manifests
```

Plan:

```
artifacts/plans/change-plan.json
```

Generated tests:

```
tests/
```

Registry:

```
src/ai/registry/test-registry.json
```

Execution results:

```
artifacts/reports
```

Feedback:

```
artifacts/healing
```

```
artifacts/flakes
```

```
artifacts/analytics
```

One final line:

The framework turns requirements into governed, traceable, validated, executable Playwright tests.